Computer Vision Analysis of Volleyball Serves

Spencer Kurtz
Department of Computer Science
Golisano College of Computing and Information Sciences
Rochester Institute of Technology
Rochester, NY 14586
sjk7876@rit.edu

***Abstract*—**

***Index Terms*—Computer vision; Volleyball analysis; Ball tracking; Sports analytics; Machine learning; YOLO; SORT; Trajectory analysis**

I. Introduction

Video analysis has become an integral part of modern sports training, offering athletes and coaches valuable feedback on performance. However, existing tools for volleyball are often limited in precision, accessibility, or scope. In particular, serve analysis remains a largely manual process, relying on subjective visual review rather than quantitative, automated measurements. This project explores how computer vision can be leveraged to develop a lightweight, single-camera system for automated serve analysis, directly from raw video.

A. Volleyball Overview

Volleyball is a fast-paced team sport that relies on precision, timing, coordination, and teamwork. Each rally starts with a serve, followed by a sequence of passes, sets, and attacks that aim to ground the ball on the opponent's court. A standard indoor court measures 18 by 9 meters, with a net that is 2.43 meters high in men's play and 2.24 meters high in women's play. Teams rotate through six positions, with each player alternating between serving, receiving, and front-row attacking roles throughout a match.

Despite its simple appearance, volleyball is defined by highly technical movements. Serves, spikes, and passes all depend on millisecond-level timing and consistent mechanics. This makes volleyball an interesting challenge for computer vision: tracking a small, fast-moving ball through occlusions, varied lighting, and motion blur while also understanding spatial context on the court. The presence of players moving across the frame, partially obscuring the ball, adds further complexity to automated analysis.

B. Importance of the Serve

The serve is the only action in volleyball that is completely under one player's control, making it one of the most critical components of the game. Each player on the court is guaranteed to serve at least once per set, and serves can directly score points if they land on the opponent's court. A strong serve can pressure the opponent's receive formation, while a weak or inconsistent one often results in free points for the opponent. Because every rally begins with a serve, it often dictates early momentum and sets the tone for subsequent plays.

Serving is both a mechanical and strategic skill. A consistent serve applies pressure by forcing predictable passes, thereby limiting the opponent's offensive options. Advanced servers vary spin, velocity, and placement to disrupt rhythm or target specific receivers. Different serve types, such as float, topspin, and jump serves, produce distinct flight paths based on their spin characteristics. A float serve, struck with minimal rotation, moves erratically through the air due to turbulent airflow, making it harder to predict. In contrast, a topspin serve generates a sharp downward curve, trading unpredictability for control and speed. Mastering both types requires precise timing of the toss, controlled contact, and consistent follow-through.

Consistency and accuracy are essential: even small deviations in toss height, contact angle, or foot position can cause the ball to drift or miss the target zone. Each serve can move at speeds up to 80 miles per hour, landing on the opponent's court in less than one second. Because the window for error is so small, even experienced players rely heavily on video feedback to refine their technique. However, traditional video review has limitations, providing only qualitative feedback.

C. Motivation for Automated Analysis

While video review is standard in volleyball training, it is still mostly done manually: slow, subjective, and inconsistent. Athletes often pause and replay clips frame by frame to estimate toss height, landing location, or flight path, but there is no consistent quantitative feedback. This makes it difficult to measure improvement or compare serves between players. Even with slow-motion playback, estimating exact ball paths or landing zones is imprecise and highly dependent on the observer's perception.

Some modern tools, such as BallTimeAI, have started automating volleyball video analysis using proprietary AI models. However, these systems are typically closed-source, subscription-based, and optimized for match-level statistics rather than detailed motion or serve-level analysis. They often focus on game outcomes, such as rally length or attack success rate, rather than the individual skills. Moreover, their reliance on specialized equipment or multiple camera angles limits accessibility for athletes and coaches who typically record sessions with a single baseline camera.

Computer vision offers a way to bridge the gap. With reliable ball tracking and court calibration, it becomes possible to automatically extract serve metrics such as toss consistency, trajectory shape, and landing accuracy straight from normal video. The goal is not to replace coaches, but to make feedback faster, objective, and repeatable.

Developing such a system also presents technical challenges. The ball's small size, fast speed, and motion blur make detection difficult, especially with a single viewpoint. Background clutter, changing lighting, and occlusion from players crossing the frame introduce further noise. Achieving accurate tracking and trajectory estimation under these conditions requires models capable of understanding both temporal motion and spatial structure. Overcoming these challenges is central to this project's motivation.

D. Project Scope and Objectives

This project focuses on building a single-camera system for automatic serve analysis using computer vision. The goal is to detect, track, and evaluate volleyball serves from standard baseline footage without any specialized hardware. By combining modern detection, segmentation, and calibration techniques, the system aims to extract meaningful performance metrics directly from video.

The scope includes five main objectives:

- **Ball Detection and Tracking:** Train and evaluate a YOLOv8-based model to detect and track the volleyball across frames during the serve.
- **Court Line Segmentation:** Use a U-Net architecture to segment court lines and generate a homography map for pixel-to-meter calibration.
- **Serve Event Segmentation:** Automatically isolate individual serve sequences (toss, flight, landing) from raw session footage.
- **Trajectory and Consistency Analysis:** Predict and visualize the ball's 2D trajectory, landing zone, and serve consistency across multiple players and sessions.
- **Visualization and Reporting:** Create a user-friendly interface to display serve metrics and visualizations for coaches and athletes.

Together, these components form a baseline system for automated serve evaluation: providing objective, repeatable feedback that can support coaching, training, and future sports analytics research. All models will be trained and tested on a custom dataset of 50-100 serves collected across multiple players and recording sessions. Beyond this initial implementation, the system can serve as a foundation for broader extensions, such as 3D trajectory reconstruction, automatic court calibration across different gym environments, and real-time serve feedback applications.

II. Related Work

III. Dataset and Annotation

A. Recording Setup

To capture consistent and analyzable volleyball serves, all recordings were conducted using a fixed baseline camera setup designed to clearly capture the server, ball trajectory, and full court. The camera was positioned at the center of the baseline, approximately 4 m behind the court, mounted on a tripod at a height of 74 in (188 cm). This configuration ensured that both the serve toss and landing zone were visible within the same frame while minimizing parallax distortion. The camera was tilted downward by approximately 5–10°, allowing the near and far court lines to remain visible without excessive floor coverage.

All sessions were recorded at the Rochester Institute of Technology Hale-Andrews Student Life Center volleyball courts under overhead LED lighting. While lighting conditions varied slightly between sessions due to court availability, no recordings required artificial exposure adjustment or post-processing corrections. The only camera setting modification was disabling the phone's built-in video stabilization to prevent frame warping and maintain consistent spatial alignment across frames.

Recordings were captured at 1920×1080 resolution and 60 frames per second (FPS) using a Samsung Galaxy S23. The 60 FPS frame rate helped minimize motion blur during high-speed serves, which was crucial for accurate ball detection and trajectory analysis. The tripod distance of 4 m was measured and kept consistent across sessions to ensure uniform perspective and homography mapping later in the pipeline. Each recording session consisted of one or more video clips, each under 15 minutes in duration, which were transferred directly to a desktop computer for processing and annotation.

No external calibration markers were used during recording; instead, court corners and boundary lines visible in the footage were later used for homography-based calibration. This choice simplified setup while still allowing accurate mapping between image coordinates and real-world court positions.

This setup produced a consistent, reproducible viewpoint across all sessions and players, providing a standardized dataset for model training and evaluation in serve detection, trajectory estimation, and landing prediction tasks.

B. Data Organization

All of the collected data was organized into a standardized directory structure to maintain consistency across sessions, players, and annotation stages. This structure was designed to support both short-term experimentation and long-term reproducibility, ensuring that each stage of the pipeline, from raw recordings to labeled training data, could be traced and reconstructed if needed. The dataset directory (data/) contains all session recordings, extracted frames, annotations, and metadata required for model training and evaluation. At the highest level, the project directory includes five main components: raw and processed videos, extracted frames, annotations, metadata, and training subsets for the models. Each subdirectory follows a uniform naming convention based on session and serve identifiers, allowing for clear correspondence between files and metadata entries.

The raw/ subfolder stores the original session footage as captured by the baseline camera, organized by recording date

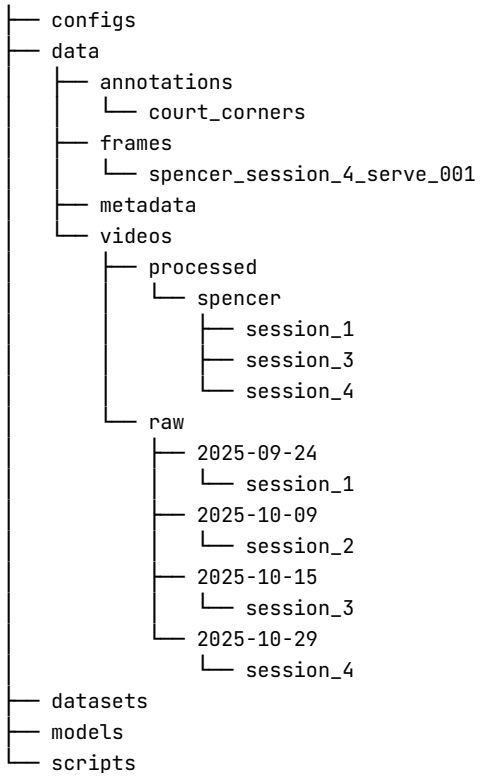

Fig. 1. Directory structure.

and session number (e.g., 2025-09-24/session_1/). Each raw session folder contains one or more .mp4 files directly transferred from the recording device. These raw videos remain unmodified and serve as the master data source for all later stages of processing. Each raw video contains only one player at a time, though individual sessions can include multiple players overall.

Processed serve clips are stored under the processed/ subfolder, which contains serve-level video segments extracted from the raw sessions using the split_serves.py utility. Each player has a dedicated folder, with sessions nested inside. Serve clips follow a consistent naming convention (serve_001.mp4, serve_002.mp4, etc.) to reflect their chronological order within each session. This hierarchy provides a clean separation between players, sessions, and individual serves while allowing straightforward iteration during model inference and annotation. Across all four sessions, a total of 113 serves were extracted and processed into individual clips from six players and four different balls, capturing a range of serving styles while maintaining a consistent visual perspective.

These numbers will change later.

To support frame-level annotation for both ball detection and court segmentation, as well as model training, all serve clips were decomposed into individual frames stored under data/frames/. Frame extraction was performed using the extract_frames.py script, which automatically matches the frame rate of the source video, preserving the temporal fidelity of the ball trajectory and motion blur characteristics. Across all sessions, a total of over 16,000 frames were extracted. These images form the basis for both bounding-box annotations used in YOLOv8 ball detection training and binary masks used for court segmentation tasks.

The data/annotations/ directory stores all manually and semi-automatically generated labels related to both the ball and the court. Each annotation type corresponds to a distinct subfolder: binary masks for court line segmentation, YOLO-format bounding boxes for ball detection, and JSON files for court corner calibration. Court calibration data are stored under data/annotations/court_corners/ as one JSON file per session, each containing six manually labeled intersection coordinates corresponding to visible court corners in the recording. Ball annotations were created using Computer Vision Annotation Tool (CVAT)[1], which allowed frame-by-frame bounding box labeling with integrated YOLO export. The annotations were stored in CVAT and later moved into the dataset for model training. The exported labels follow the YOLO text format, with one .txt file per frame. This structure integrates directly with the Ultralytics YOLOv8 framework and supports efficient data augmentation during training.

Complementing the visual and annotation data, the data/metadata/ directory provides structured metadata linking sessions, players, and serves. Two CSV files, players.csv and serves.csv, serve as lightweight relational tables for cross-referencing dataset components. players.csv contains one row per player, listing the player's identifier, name, handedness, and any relevant notes such as skill level or role (e.g., varsity, club, etc.). serves.csv is the primary index for all serve clips in the dataset. Each row records the player name, serve ID, source video path, processed output path, serve start and end frames, and the annotated landing frame. These fields are automatically updated by the serve-splitting and landing-labeling scripts (split_serves.py and landing_frame.py), ensuring consistency between raw recordings, processed serve clips, and corresponding annotations.

All data and configuration files are maintained under version control using Git, ensuring that dataset revisions, annotation corrections, and script updates remain tracked over time. This practice is particularly important for reproducibility in experimental pipelines involving iterative model training and evaluation. Large binary files such as videos and frame sequences are tracked through external storage references to prevent repository bloat, while annotation files and metadata remain directly versioned for transparency. The clear directory structure also facilitates automated dataset formatting and validation. Utility scripts such as format_datasets.py and auto_label_and_upload.py rely on this consistent organization to locate and process files without manual path specification. By enforcing standardized folder names and file formats, the dataset remains scalable and compatible with future extensions, including additional players, camera angles, or labeling types.

C. Annotation Process

The annotation process established the foundation for all later stages of serve analysis, providing the spatial and temporal ground truth needed for model training, evaluation, and trajectory reconstruction.

All labeling was conducted in a semi-automated workflow using the open-source CVAT alongside custom Python scripts for data organization and post-processing. CVAT provided an interactive web interface that supported both manual and interpolated bounding box placement, while the scripts handled export conversion and metadata integration.

For ball detection, each serve clip was uploaded to CVAT, where bounding boxes were drawn around every ball on a frame-by-frame basis. CVAT's interpolation and automatic tracking features were used to propagate bounding boxes across consecutive frames, substantially reducing manual effort while maintaining spatial precision. After annotation, the labels were exported directly in YOLO format, producing one .txt file per frame with normalized center coordinates and bounding-box width and height. These labels were stored alongside the extracted frames within each serve's directory to match the YOLOv8 training input structure.

After training intermediate YOLOv8 ball detection models, they were used to semi-automate the annotation process. The model was used to output predictions for the clip frames, which were uploaded to CVAT, greatly reducing the amount of time spent manually adjusting bounding boxes. These predictions were then verified before being completed and used for further training.

Serve landing frames were annotated separately using the landing_frame.py utility. Each serve clip was reviewed manually, and the frame index where the ball first contacted the court was recorded in the serves.csv metadata file under the landing_frame column. This temporal annotation enables direct evaluation of predicted landing times and supports future trajectory modeling tasks.

Finally, court geometry was annotated once per recording session using the annotate_court.py script. Each JSON file in data/annotations/court_corners/ contains four corner coordinates and each side of the center line, used to compute a homography matrix mapping image coordinates to real-world court positions. This calibration step ensures geometric consistency across all serves recorded under the same setup and allows later visualizations, such as landing zone maps, to be expressed in physical units rather than pixels.

Together, these annotation procedures established a unified, verified dataset linking ball position, timing, and court geometry. The resulting labels formed the backbone of the detection, tracking, and trajectory analysis pipeline developed in this project.

This turned into a lot of writing that I'll reduce later.

IV. Methodology

A. Ball Detection (YOLOv8)

B. Ball Tracking

C. Court Segmentation (U-Net)

D. Court Calibration (Homography)

E. Trajectory & Landing Analysis

V. Evaluation

A. YOLOv8 detection metrics

B. U-Net segmentation accuracy

C. Tracking quality

D. Homography precision

E. Serve consistency

VI. Results

A. Quantitative performance summary

B. Example visualizations

VII. Future Work

A. Limitations

B. Future Improvements

Acknowledgment

The authors would like to thank...

References

[1] C. Corporation, "Computer vision annotation tool (cvat)," Jun. 2023. [Online]. Available: https://doi.org/10.5281/zenodo.8070041